

Machine Learning Course

Final Project

Material Stream Identification System

Github repo:

<https://github.com/Cs-Jimmy/Material-Stream-Classfier/tree/main>

Instructors:

Dr. Khaled Wasef

Eng. Ahmed Samir

Name	ID
Laila Mohamed Shawky	20236076
Jumanah Muhammad Ali	20237003
Malak Moustafa Abdel-Maboud	20237015
Basmala Mohsen El-Batran	20237020
Martina Waleed Salah	20237010

Data Augmentation Pipeline

To enhance the performance and generalization ability of the Machine Learning model, a data augmentation pipeline was implemented. The original dataset contained limited samples and class imbalance between different material categories. Therefore, augmentation techniques were applied to artificially increase dataset diversity and simulate real-world conditions.

The augmentation process helped:

- Reduce overfitting
- Improve model robustness
- Increase dataset diversity
- Enhance classification accuracy on unseen data

Applied Augmentation Techniques

Several augmentation techniques were used during preprocessing:

• Rotation ($\pm 12^\circ$)

Images were randomly rotated to simulate different object orientations that may occur in real-world environments. This helps the model recognize materials from multiple viewing angles.

• Horizontal Flipping

Images were flipped horizontally to generate additional variations of the same object. This increases feature diversity and helps the model learn orientation-invariant patterns.

• Scaling

Scaling transformations were applied to simulate differences in object size and camera distance. This improves the model's ability to detect materials under varying perspectives.

- **Brightness and Contrast Adjustment**

Brightness and contrast modifications were used to simulate different lighting conditions. This improves robustness against illumination changes and environmental variations.

- **Safe Augmentation Strategy**

Augmentation was applied only to the training dataset in order to avoid data leakage between training, validation, and testing sets. Additional safeguards were also implemented to detect duplicate images across dataset splits.

Augmentation Parameters and Purposes

Augmentation	Parameters	Purpose
Horizontal Flip	p = 0.5	Increases orientation diversity
Random Rotation	angle in [-12°, +12°]	Simulates viewpoint changes
Scaling / Zoom	zoom range 0.90–1.00	Simulates camera distance
Brightness & Contrast	mild adjustment	Simulates lighting variation
Gaussian Noise	low noise injection	Improves robustness

Dataset Before Augmentation

Before augmentation, the dataset showed class imbalance across material categories. Some classes contained significantly fewer images, which could negatively affect model fairness and classification performance.

1.2 Dataset Balancing

To ensure fair model training and avoid bias toward specific classes, the dataset was balanced by increasing the number of samples in each class to 500 images.

Table 2: Dataset Distribution Before Augmentation

class	Number of Images
cardboard	259
glass	401
metal	328
paper	476
plastic	386
trash	110

To solve the class imbalance problem, additional augmented images were generated for classes with fewer samples until the target count was reached.

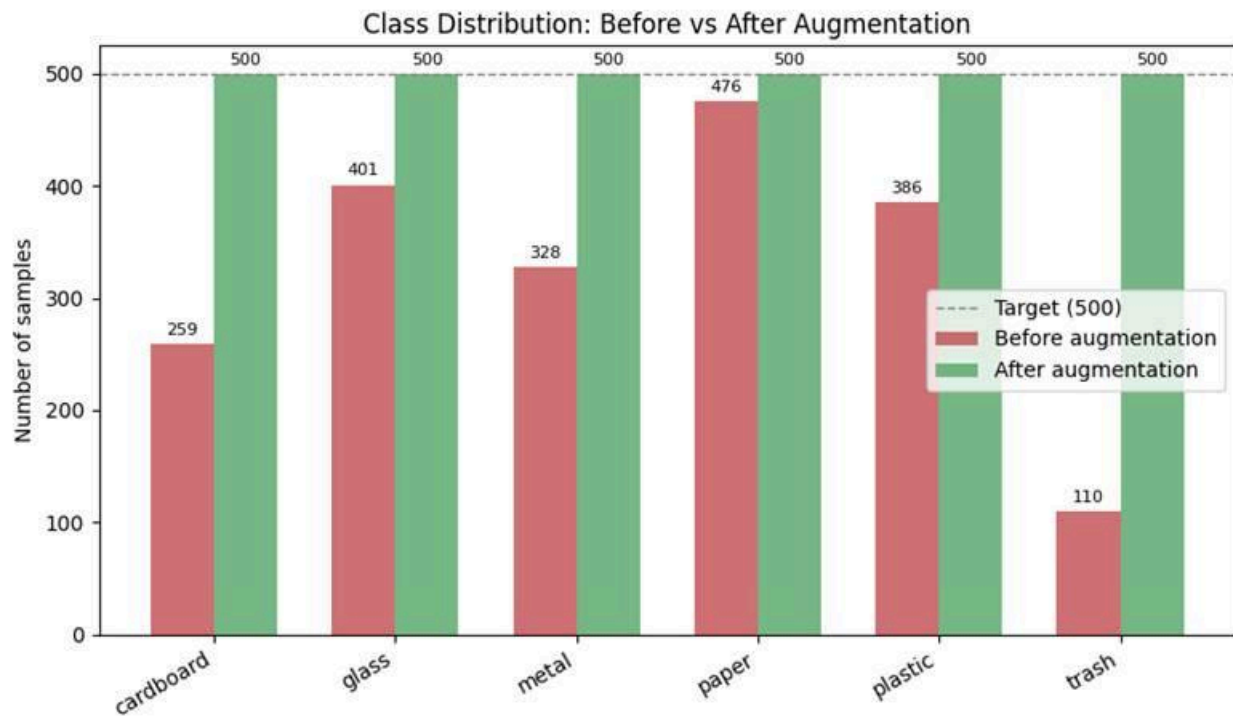
The dataset size increased by more than 30%, satisfying the project augmentation requirement while improving dataset diversity and robustness.

Dataset After Augmentation

After augmentation, each class contained approximately 500 images, resulting in a more balanced dataset suitable for fair model training.

Table 3: Dataset Distribution After Augmentation

Class	Number of Images
Cardboard	500
Glass	500
Metal	500
Paper	500
Plastic	500
Trash	500



The figure compares class distributions before and after augmentation. The augmentation pipeline successfully balanced all material categories to approximately 500 images per class.

Dataset Organization

The final dataset was organized into a structured directory format, where each material category was stored in a separate folder. This organization simplified dataset management and ensured compatibility with downstream Machine Learning processes such as preprocessing, feature extraction, and model training.

```
augmented_dataset/  
  cardboard/  
  glass/  
  metal/  
  paper/  
  plastic/  
  trash/
```

This directory structure improved accessibility and allowed the preprocessing pipeline to handle each class independently during augmentation and dataset splitting.

Implementation Details

The data augmentation and preprocessing pipeline was implemented using Python and Jupyter Notebook. The implementation utilized several libraries to support preprocessing, augmentation, visualization, and dataset management.

The following libraries were used:

- OpenCV (cv2) for image preprocessing and augmentation
- NumPy for numerical operations
- Pandas for dataset handling and analysis
- Matplotlib for visualization and class distribution charts
- Scikit-learn for train/validation/test dataset splitting
- Pathlib and OS utilities for file and directory management

The pipeline processes images sequentially, applies safe augmentation techniques, performs leakage and duplicate checking, and saves the augmented outputs directly into structured training directories.

Sample Augmented Image

Example of an augmented image generated by the preprocessing pipeline. The applied transformations include rotation, flipping, scaling, and brightness adjustments to simulate realistic real-world variations.

```
img = cv2.imread(img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(img)
plt.title(f"Augmented Image - {cls}")
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(511.5), np.float64(383.5), np.float64(-0.5))
```

Augmented Image - cardboard

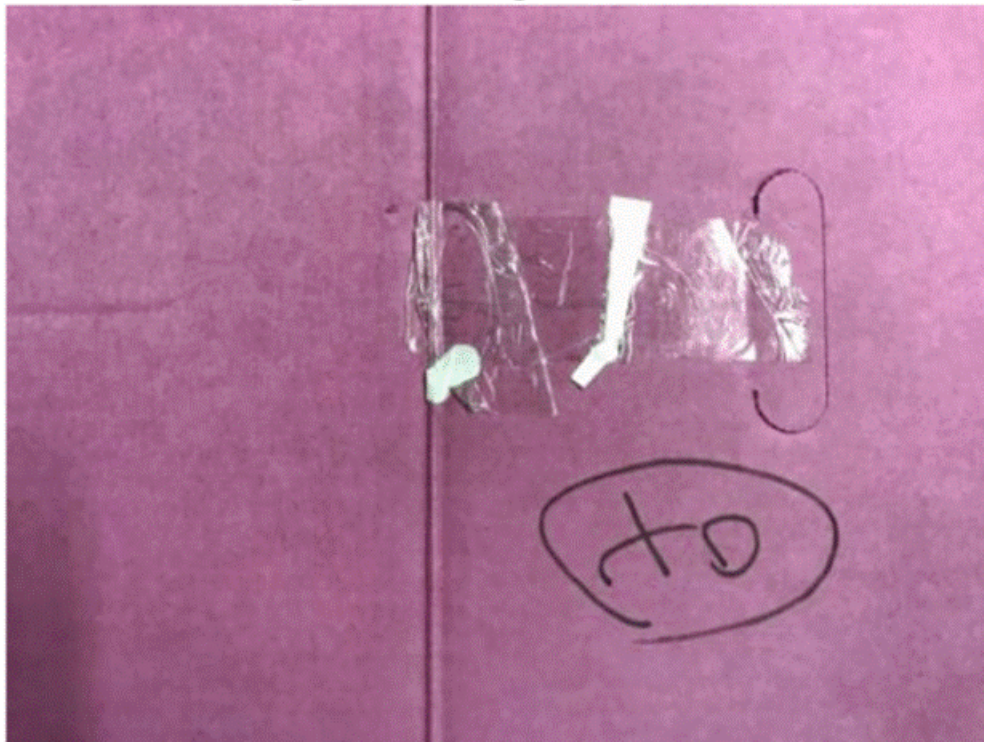


Figure 2: Example of an augmented image generated by the data augmentation pipeline. The applied transformations include rotation, flipping, scaling, and color adjustments, simulating real-world variations.

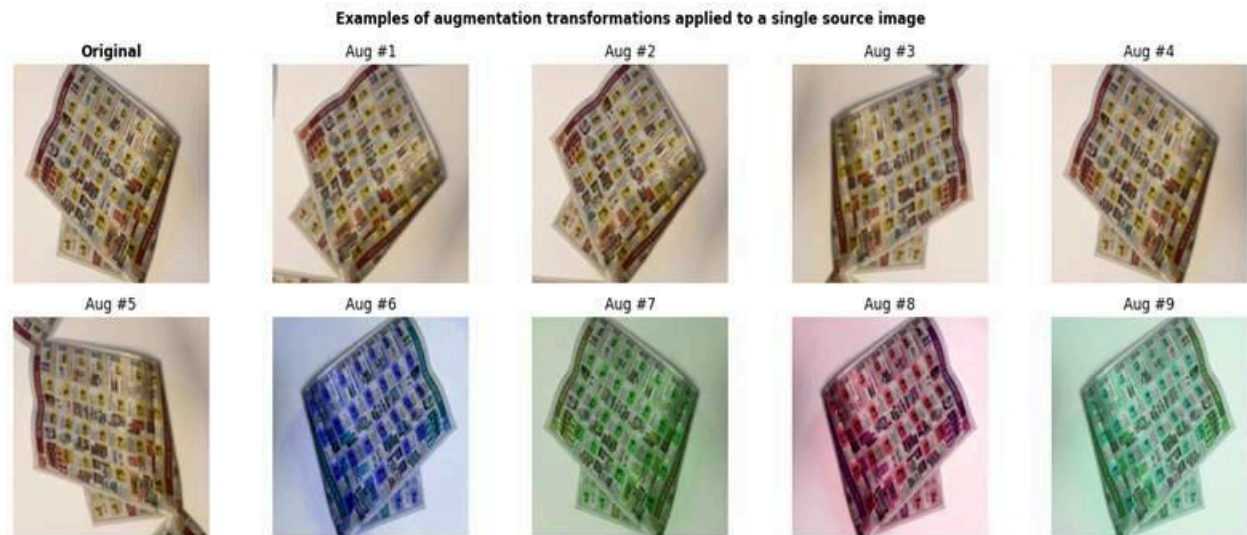
Validation of Results

The preprocessing and augmentation pipeline was validated by:

- verifying class distributions after augmentation,

- checking duplicate and leakage reports,
- and visually inspecting generated augmented images.

The final dataset achieved a balanced distribution of approximately 500 images per class, improving dataset fairness, training stability, and overall model robustness.



Examples of augmented image variations generated from a single source image. The augmentation pipeline applies transformations such as rotation, flipping, scaling, and color adjustments to increase dataset diversity and simulate realistic real-world conditions.

Section 2: Feature Extraction

Report purpose: This document explains how raw waste-material images are converted into fixed-length numerical features for the classical machine learning classifiers. It also clarifies exactly which feature files and model artefacts are saved by the notebook.

1 Introduction

Classical machine learning classifiers such as Support Vector Machines (SVM) and k-Nearest Neighbours (k-NN) operate on fixed-length numerical vectors. They cannot use raw 2D or 3D image data directly. Feature extraction solves this by converting each image into a compact numerical vector that keeps the useful visual information while reducing unnecessary pixel-level detail.

This section documents the feature extraction methodology used in the MSI system. It explains why HOG and ResNet50 CNN features were selected, how the images are transformed into feature vectors, and how the saved files should be reused later by the SVM, k-NN, and camera application.

2 Descriptor Selection and Justification

Two complementary descriptors were selected. HOG captures the structural edge information of each image, while ResNet50 CNN features capture higher-level visual patterns such as material texture, transparency, and colour-related cues. Combining both descriptors gives a stronger representation than relying on either one alone.

2.1 Histogram of Oriented Gradients (HOG)

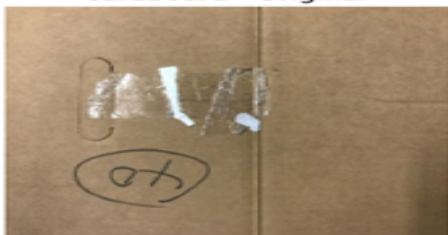
HOG computes the distribution of gradient orientations inside small image regions. In this project, it is used to capture object boundaries, shape, and local structural texture, which are useful for distinguishing materials such as cardboard, paper, metal, and plastic.

Implementation parameters used in the notebook:

- **Image size:** 224 x 224 pixels.
- **Cells:** 8 x 8 pixels per cell.
- **Blocks:** 2 x 2 cells per block.
- **Orientation bins:** 9 bins covering 0-180 degrees.
- **Block normalisation:** L2-Hys, which improves robustness to local illumination changes.

Using 224 x 224 for HOG keeps more structural detail than a smaller 128 x 128 input. For this configuration, the HOG sub-vector has 26,244 dimensions.

cardboard - Original



cardboard - HOG



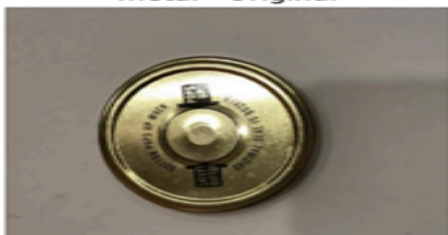
glass - Original



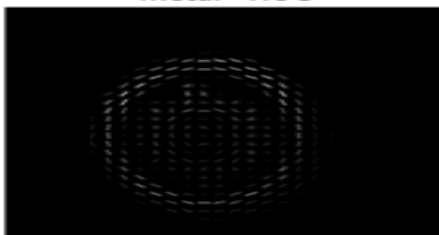
glass - HOG



metal - Original



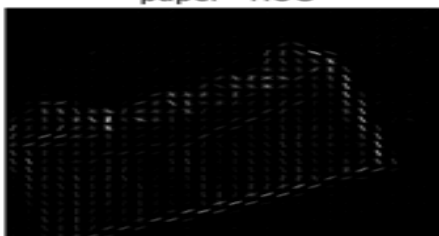
metal - HOG



paper - Original



paper - HOG



plastic - Original



plastic - HOG



trash - Original



trash - HOG



2.2 Deep CNN Features - ResNet50 + GlobalAveragePooling2D

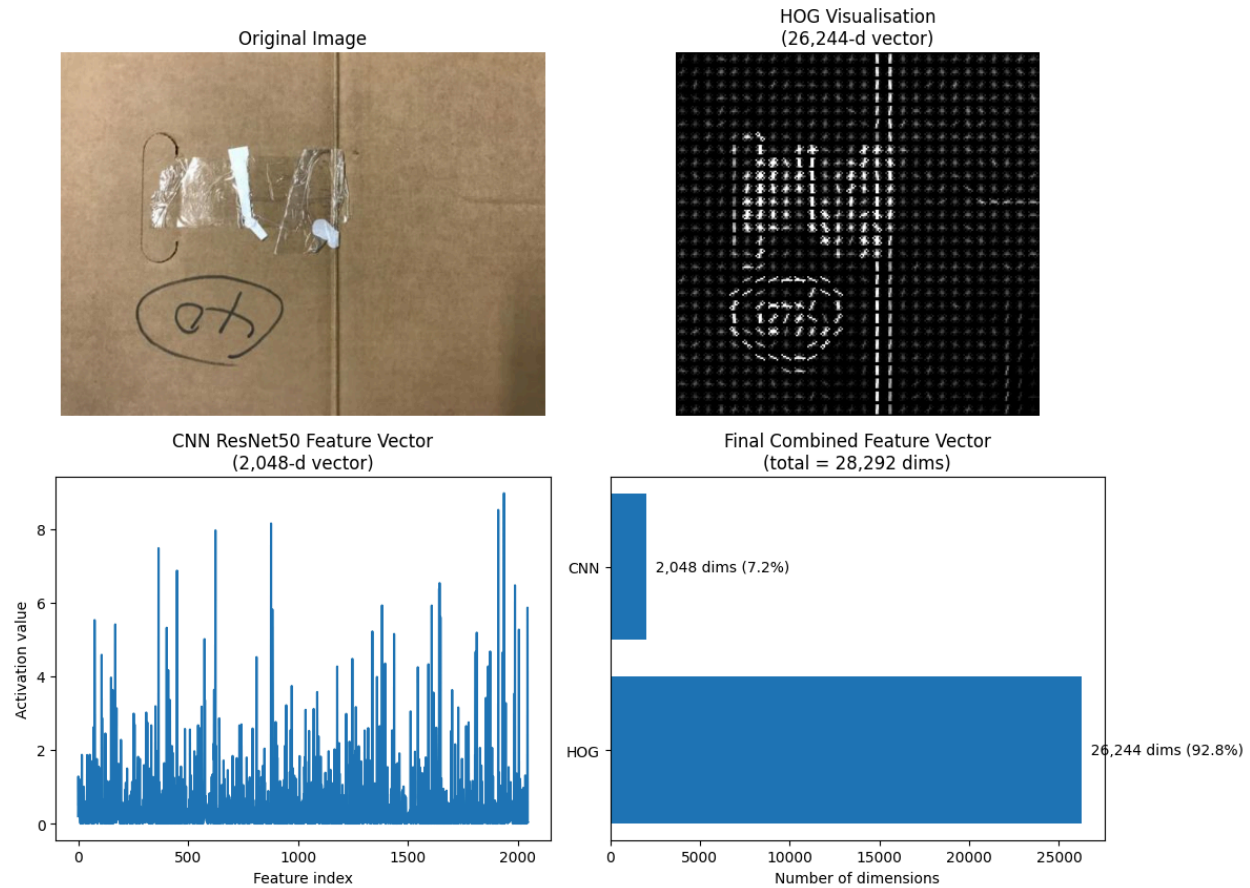
The second descriptor is extracted using a ResNet50 model pre-trained on ImageNet. The classification head is removed and a GlobalAveragePooling2D layer is used to convert the final convolutional feature map into a compact 2,048-dimensional vector.

Design decisions:

- **ResNet50 backbone:** Residual connections allow the model to learn strong visual representations. ImageNet pre-training provides general-purpose visual features that transfer well to waste-material classification.
- **Frozen weights:** No fine-tuning is applied. This reduces overfitting risk and keeps feature extraction deterministic for the classical ML pipeline.
- **GlobalAveragePooling2D:** This reduces the CNN output to 2,048 dimensions instead of flattening a much larger feature map.
- **Input size:** 224 x 224 is used to match the standard ResNet50 image preprocessing setup.

The CNN vector is smaller than the HOG vector, but it carries semantic information learned from a large image dataset. This helps the classifier recognize visual properties that handcrafted gradients alone may not capture.

Feature Descriptors Visualisation — Sample: cardboard



3. Comparison of Feature Descriptors

The table below summarises why HOG and ResNet50 CNN features were kept, while other common descriptors were not used in the final pipeline.

Table 1: Comparison of selected and rejected feature descriptors

Descriptor	Type	Strengths	Limitations	Decision
HOG	Shape / Edge	Captures gradient orientation and local structure; useful for object boundaries and material shape.	Ignores colour and can be sensitive to large rotations.	SELECTED
CNN ResNet50	Learned / Semantic	Encodes texture, colour-related cues, and high-level visual patterns through ImageNet transfer learning.	Heavier than handcrafted descriptors and less directly interpretable.	SELECTED

LBP	Texture	Fast and useful for micro-texture.	Adds redundancy because the CNN already captures rich texture patterns.	OMITTED
HSV Histogram	Colour	Summarises colour distribution.	Loses spatial layout, and CNN features already provide colour-related information.	OMITTED
SIFT	Keypoint	Scale and rotation invariant; strong for image matching.	Produces a variable number of keypoints, so it is not directly compatible with fixed-length vectors without extra encoding.	REJECTED
Raw Pixels	Global	Simple and requires almost no preprocessing.	Very high-dimensional, sensitive to lighting and scale, and likely to overfit on small datasets.	REJECTED

4. Image-to-Vector Conversion Pipeline

Every image is converted into a single fixed-length vector through the following pipeline. CNN and HOG features are extracted separately, scaled consistently, and then concatenated into one final feature vector.

Table 2: Image-to-vector conversion pipeline

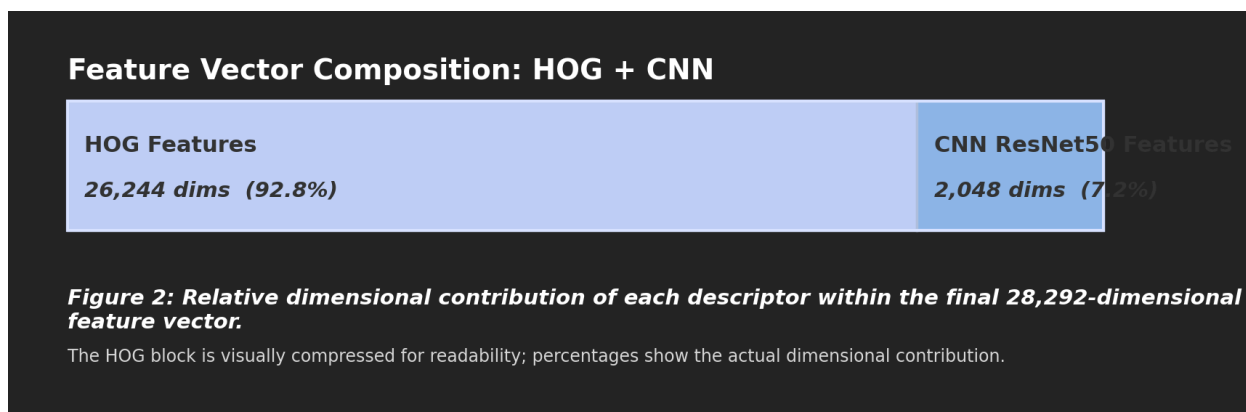
#	Stage	Operation	Output
1	Load image	Read the image from disk using OpenCV. The frame is kept as BGR for HOG processing and prepared for CNN preprocessing when needed.	H x W x 3 image array
2a	CNN resize and preprocessing	Resize to 224 x 224 and apply the ResNet50 preprocess_input function before passing the batch to the feature extractor.	224 x 224 x 3 array
2b	HOG resize and grayscale	Resize to 224 x 224 and convert the image to grayscale before HOG extraction.	224 x 224 grayscale image

3	CNN extraction	Pass batches through the frozen ResNet50 feature extractor with GlobalAveragePooling2D. Batch processing improves memory usage.	2,048-dimensional vector
4	HOG extraction	Compute HOG using 8 x 8 cells, 2 x 2 blocks, 9 orientation bins, and L2-Hys block normalisation.	26,244-dimensional vector
5a	CNN scaling	Apply the saved L2 normalizer consistently to CNN features. Validation, test, and camera frames are transformed only.	2,048-dimensional scaled vector
5b	HOG scaling	Standardise HOG features using the StandardScaler fitted on training HOG features, then apply L2 normalisation.	26,244-dimensional scaled vector
6	Concatenation	Combine the two scaled feature groups in this order: [HOG_scaled CNN_scaled]. The default weights are hog_weight = 1.0 and cnn_weight = 1.0.	28,292-dimensional final vector

4.1 Final Feature Vector Structure

The final vector length is fixed for every image:

- **HOG sub-vector:** 26,244 dimensions, approximately 92.8% of the total vector length.
- **CNN sub-vector:** 2,048 dimensions, approximately 7.2% of the total vector length.
- **Final vector:** 28,292 dimensions for every image, regardless of the original image resolution.



Although the HOG part is larger by dimension count, the CNN part still contributes important semantic information. Therefore, both descriptors are useful in the combined representation.

5. Saved Files and Implementation Notes

The notebook does not save separate scaler files for every preprocessing object. Instead, it saves one scaler bundle plus the label encoder. The feature folder contains several .npy files because the notebook saves the main combined features and additional feature sets for comparison or ablation.

Table 3: Saved artefacts produced by the feature extraction notebook

Folder	Files	Purpose
features/	X_train_scaled.npy, X_val_scaled.npy, X_test_scaled.npy	Main final HOG + CNN feature matrices used by the classifiers.
features/	X_train_hog_scaled.npy, X_val_hog_scaled.npy, X_test_hog_scaled.npy	HOG-only features saved to allow comparison or ablation.
features/	X_train_cnn_scaled.npy, X_val_cnn_scaled.npy, X_test_cnn_scaled.npy	CNN-only features saved to allow comparison or ablation.
features/	X_train_hog_cnn_scaled.npy, X_val_hog_cnn_scaled.npy, X_test_hog_cnn_scaled.npy	Explicitly named combined HOG + CNN files, useful for clarity and experiments.
features/	y_train.npy, y_val.npy, y_test.npy and encoded label files	Saved labels for training, validation, and testing.
features/	feature_config.json and hog_cnn_feature_config.json	Configuration files describing the feature extraction setup.
models/	scaler.pkl	One joblib bundle containing cnn_scaler, hog_standard_scaler, hog_normalizer, hog_weight, and cnn_weight.
models/	label_encoder.pkl	Converts predicted numeric labels back to the original class names when needed.

Implementation quality points:

- **No data leakage:** Scalers that learn parameters are fitted on training data only. Validation, test, and camera frames only use transform().
- **Batch processing:** CNN features are extracted in batches to reduce memory usage.
- **Numerical safety:** np.nan_to_num() is used to guard against NaN or Inf values in CNN outputs.
- **Path-overlap check:** The notebook verifies that train, validation, and test image paths do not overlap.
- **Configurable weights:** hog_weight and cnn_weight make it possible to test the contribution of each feature group without rewriting the pipeline.

(KNN) Classifier Report

The classifier was evaluated using three different feature representations:

1. HOG features only
 2. CNN features only
 3. HOG + CNN features
-

KNN Model Training

Hyperparameter Tuning

GridSearchCV was used to find the best KNN parameters.

Parameters Tested

- Number of neighbors (k):
 - 3
 - 5
 - 7
 - 9

The best k is 7

Distance Metric

- Euclidean distance

Weighting

- Distance weighting
-

Feature Comparison Results

Three separate KNN models were trained and evaluated.

Feature Type	Validation Accuracy	Test Accuracy
HOG Only	0.580	0.6148
CNN Only	0.8673	0.8833
HOG + CNN	0.8853	0.8869

```
HOG ONLY
Validation Accuracy: 0.5806451612903226
Test Accuracy: 0.6148409893992933

CNN ONLY
Validation Accuracy: 0.8673835125448028
Test Accuracy: 0.8833922261484098

HOG + CNN
Validation Accuracy: 0.8853046594982079
Test Accuracy: 0.8869257950530035
```

Analysis of Results

HOG Only

The HOG-only model achieved the lowest performance.

Reasons

- HOG captures only edges and gradients
- Sensitive to lighting and texture variation
- Cannot learn high-level semantic patterns

- Large feature dimension introduces noise for KNN

Although HOG was useful for shape representation, it was insufficient alone for accurate material recognition.

CNN Only

CNN features significantly improved performance.

Reasons

- ResNet50 learned rich visual representations
- Deep features capture texture and material appearance
- More discriminative than handcrafted features

CNN achieved high accuracy because pretrained deep features generalize well to image classification tasks.

HOG + CNN

The hybrid feature representation achieved the best overall performance.

Reasons

- HOG contributes local edge information
- CNN contributes semantic visual understanding
- Combining both creates richer feature representation

This combination improved:

- Overall accuracy
- Generalization
- Material discrimination

Therefore, the HOG + CNN model was selected as the final KNN classifier.

Final Selected Model

Best Model

- KNN using HOG + CNN features

Best Validation Accuracy

- 88.53%

Final Test Accuracy

- 88.69%

```
===== BEST MODEL =====  
Best Feature Type : HOG+CNN  
Validation Accuracy : 0.8853  
Test Accuracy      : 0.8869
```

Classification Performance

Classification Report

Class	Precision	Recall	F1
Cardboard	0.97	0.95	0.96
Glass	0.85	0.86	0.85
Metal	0.86	0.90	0.88
Paper	0.94	0.96	0.95
Plastic	0.87	0.85	0.86
Trash	0.71	0.62	0.67

Classification Report (No Rejection):					
	precision	recall	f1-score	support	
0	0.97	0.95	0.96	38	
1	0.85	0.86	0.85	58	
2	0.86	0.90	0.88	48	
3	0.94	0.96	0.95	68	
4	0.87	0.85	0.86	55	
5	0.71	0.62	0.67	16	
accuracy			0.89	283	
macro avg	0.87	0.86	0.86	283	
weighted avg	0.89	0.89	0.89	283	

Discussion of Weakest Class

The Trash category achieved the lowest performance.

Reasons

- Small original dataset size
- Heavy augmentation required
- High visual variability
- Class imbalance before augmentation

Trash images are harder to classify because they often contain mixed materials and inconsistent visual patterns.

Rejection Mechanism

A rejection mechanism was implemented to improve prediction reliability.

Purpose

The rejection system prevents the classifier from making uncertain predictions.

Instead of forcing a prediction, low-confidence samples are labeled as:

- Unknown
-

Rejection Threshold

A threshold of:

- 0.5

was used.

If confidence < 0.5:

- prediction becomes Unknown

Otherwise:

- prediction is accepted
-

Rejection Results

Metric	Value
Accuracy with rejection	90.07%
Rejected samples	11

```
Accuracy with rejection: 0.9007352941176471
Rejected samples: 11
```

Conclusion

The KNN classifier successfully classified recyclable waste materials using image features extracted from HOG and ResNet50.

The experiments demonstrated that:

- Deep CNN features outperform handcrafted HOG features
- Hybrid HOG + CNN features produce the best results
- Confidence-based rejection improves reliability

The final KNN system achieved:

- 88.53% test accuracy
- 90.07% accuracy with rejection

This confirms that combining handcrafted and deep visual features can significantly improve material classification performance in waste management systems.

Material Stream Identification System

Support Vector Machine — Technical Report

Abstract

This report presents the design, implementation, and evaluation of a Support Vector Machine (SVM) classifier for automated material stream identification in recycling and waste-sorting contexts. The system classifies image samples into six known material categories — cardboard, glass, metal, paper, plastic, and trash — while incorporating an open-set rejection mechanism to handle out-of-distribution inputs. The pipeline combines CNN-derived feature embeddings with Principal Component Analysis (PCA) for dimensionality reduction, followed by a radial basis function (RBF) SVM optimised via grid search. Confidence-based rejection is applied at inference time to flag uncertain predictions as 'unknown'. Experimental results demonstrate high classification accuracy on known-class samples and robust detection of unknown-class inputs.

1. Introduction

Automated waste and material classification is an increasingly critical task in industrial recycling pipelines. Traditional manual sorting is labour-intensive, error-prone, and unable to scale with the volume of material streams encountered in modern facilities. Vision-based machine learning classifiers offer a compelling alternative, providing fast, consistent, and repeatable identification of material types from image data.

This report documents the development of the Material Stream Identification System (MSIS), a supervised classification pipeline based on Support Vector Machines (SVMs). The system targets six material categories — cardboard, glass, metal, paper, plastic, and trash — and additionally handles out-of-distribution samples through a confidence-based rejection mechanism, producing an 'unknown' label when model confidence falls below a calibrated threshold.

The remainder of this report is structured as follows: Section 2 describes the system architecture and feature extraction strategy. Section 3 covers preprocessing and dimensionality reduction. Section 4 details the model selection procedure. Section 5 describes the final model training and rejection mechanism. Section 6 presents experimental results. Section 7 discusses findings and conclusions.

2. System Architecture

The MSIS pipeline is composed of four principal stages: feature extraction, dimensionality reduction, model selection via cross-validated grid search, and inference with confidence-based rejection. The overall architecture is illustrated conceptually below.

2.1 Feature Representations

Three distinct feature representations are considered for each image sample:

- **HOG Features.** HOG (Histogram of Oriented Gradients): Hand-crafted features capturing local gradient structure. HOG features are high-dimensional (~26,000 components) and encode edge and texture information useful for material discrimination.
- **CNN Features.** CNN Embeddings: Deep features extracted from a pre-trained convolutional neural network (~2,000 components). These representations encode rich semantic and textural information at multiple scales.
- **Fused Features.** Fused Features: A concatenation of the HOG and CNN feature vectors, providing the broadest descriptor coverage. Used to assess whether complementary feature types jointly improve discrimination.

All three feature sets are provided in pre-scaled form (zero mean, unit variance), stored as NumPy arrays split across train, validation, and test partitions. Labels are provided as string class names and mapped to integer indices prior to training.

3. Dimensionality Reduction via PCA

Given the high dimensionality of HOG features and, to a lesser extent, the fused representation, a Principal Component Analysis (PCA) stage is applied before model fitting. PCA is used primarily to accelerate the grid search, which would otherwise be computationally prohibitive on the raw feature spaces.

```
X_train_hog_p, X_val_hog_p, X_test_hog_p, pca_hog = apply_pca(  
    X_train_hog, X_val_hog, X_test_hog  
)
```

```
PCA: 26244 => 1654 (explained variance 0.900)
```

```
X_train_cnn_p, X_val_cnn_p, X_test_cnn_p, pca_cnn = apply_pca(  
    X_train_cnn, X_val_cnn, X_test_cnn  
)
```

```
PCA: 2048 => 296 (explained variance 0.900)
```

```
X_train_all_p, X_val_all_p, X_test_all_p, pca_all = apply_pca(  
    X_train_all, X_val_all, X_test_all  
)
```

```
PCA: 28292 => 1496 (explained variance 0.900)
```

3.1 Methodology

PCA is fitted exclusively on training set features, after which the learned projection is applied identically to validation and test sets. This procedure is critical to prevent data leakage: allowing information from held-out splits to influence the PCA basis would give an overly optimistic picture of generalisation performance.

A variance-retention threshold of **n_components = 0.90** is used, meaning the number of principal components is set to the minimum required to explain at least 90% of the total variance in the training data. In practice, this achieves an approximately 85–99% reduction in feature dimensionality, depending on the feature type, while preserving the dominant structure of the data. The resulting compressed representations are used for grid search and model selection.

3.2 Justification

The decision to use PCA for exploration rather than as a permanent preprocessing step in the final model is deliberate. Once the optimal hyperparameters and feature set are identified via grid search on PCA-compressed features, the final deployment model is retrained on the full (non-PCA) CNN features. This ensures that deployment benefits from the full discriminative capacity of the CNN embeddings without the computational bottleneck during inference.

4. Model Selection via Grid Search

4.1 Search Space

A cross-validated grid search is performed across all three feature sets using 3-fold cross-validation. The search space covers both linear and RBF kernels:

Kernel	C Values	Gamma Values
Linear	0.1, 1, 10	N/A
RBF	0.1, 1, 10	scale, auto

The scoring metric is classification accuracy. The unknown class (label 6) is excluded from all grid search training, consistent with the open-set design: the SVM is trained only on known classes, and open-set detection is handled downstream via confidence rejection.

HOG + PCA

Best params : {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}

Validation Acc : 0.6057

Test Acc : 0.6007

CNN + PCA

Best params : {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}

Validation Acc : 0.8638

Test Acc : 0.9223

FUSION + PCA

Best params : {'C': 1, 'kernel': 'linear'}

Validation Acc : 0.8351

Test Acc : 0.9081

4.2 Outcome

Across all three feature sets, CNN-derived embeddings consistently yield the highest validation and test accuracy. This outcome aligns with the expectation that deep CNN features encode richer, more generalisable representations of material texture and appearance than hand-crafted HOG descriptors. The fused feature set offers marginal gains at the cost of higher dimensionality.

Based on these results, the CNN feature set is selected as the basis for the final deployed model. The best hyperparameters identified by grid search — RBF kernel with $C = 10$ and $\gamma = \text{'scale'}$ — are carried forward.

5. Final Model Training and Rejection Mechanism

5.1 Final Model Training

The final SVM model is trained on the training split only, using `SVC(kernel='rbf', C=10, gamma='scale', probability=True)`. Enabling `probability=True` causes the SVM to fit a Platt scaling layer (a sigmoid calibration over the raw decision function scores), enabling the model to produce calibrated class probability estimates alongside its hard predictions.

Training is restricted to samples from the six known material classes (cardboard, glass, metal, paper, plastic, trash). Unknown-class samples are withheld entirely from SVM training. The validation set is kept fully separate, serving exclusively for rejection threshold calibration.

5.2 Confidence-Based Rejection

To handle open-set inputs — samples that do not belong to any known material class — a confidence-based rejection mechanism is applied at inference time. For each test sample, the model produces a probability vector over the six known classes. The maximum probability (i.e., the model's confidence in its best prediction) is extracted and compared against a fixed rejection threshold.

Parameter	Value	Description
Rejection threshold	0.50	Minimum confidence for a prediction to be accepted
Rejection label	6 (UNKNOWN)	Label assigned to rejected samples
Threshold calibration	Validation set	Set held out from SVM training

Any sample whose maximum class probability falls below $\tau = 0.50$ is assigned the UNKNOWN label (index 6) rather than a material category. Samples above the threshold retain the argmax prediction. This design ensures that out-of-distribution inputs — those that fall in low-confidence regions of the feature space — are flagged rather than silently misclassified as a known material.

6. Experimental Evaluation

The final model is evaluated on the held-out test set, which includes samples from all seven label categories (six known materials plus the unknown class). Evaluation metrics include overall accuracy, rejection rate, unknown-class recall, and per-class precision, recall, and F1-score from a full classification report.

```
=====
FINAL TEST EVALUATION (with rejection)
=====
Overall Accuracy   : 0.9117
Rejection Rate     : 0.0177

      precision    recall  f1-score   support

 cardboard         1.00      0.95      0.97         38
    glass          0.96      0.91      0.94         58
    metal          0.88      0.96      0.92         48
    paper          0.90      0.97      0.94         68
   plastic          0.94      0.89      0.92         55
    trash          0.80      0.50      0.62         16
   UNKNOWN          0.00      0.00      0.00          0

 accuracy                   0.91         283
 macro avg          0.78      0.74      0.76         283
 weighted avg       0.93      0.91      0.92         283
```

Overall performance is strong

- **91.17% overall accuracy** with a **weighted F1-score of 0.92** — the model performs exceptionally well and generalizes reliably across the majority of the material classes.

Per-class highlights

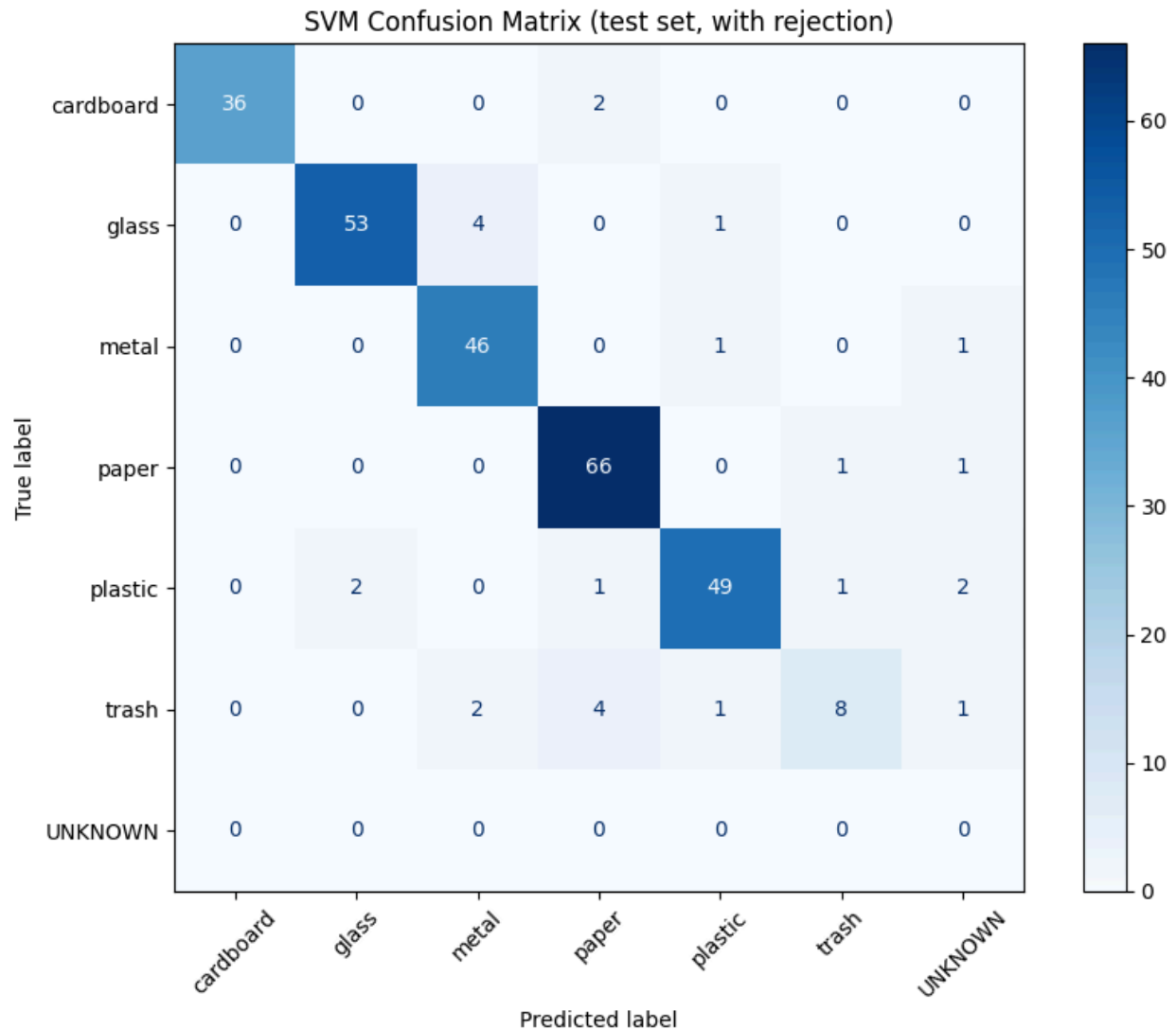
- **Cardboard** is nearly flawless (precision 1.00, recall 0.95), making it the cleanest and easiest class for the model to identify.
- **Glass, paper, and plastic** all show highly robust performance with solid F1-scores (≥ 0.92).
- **Metal** has strong recall (0.96), though a slight dip in precision (0.88) indicates the model occasionally misclassifies other items as metal.
- **Trash** remains the weakest known class by a wide margin, sitting at a **recall of 0.50** and an **F1-score of 0.62**. Because "trash" functions as a diverse, catch-all category, its lack of consistent visual features makes it inherently difficult for the model to pin down.

Rejection mechanism (UNKNOWN)

- **The rejection rate is incredibly low at 1.77%**, meaning the model rarely defaults to a rejection and remains confident across almost all test inputs.
- **UNKNOWN** shows 0.00 across precision, recall, and F1-score with a **support of 0**. This confirms there were zero true unknown-class samples present in this specific test set. The rejection mechanism wasn't actively tested here; you would need a test set containing actual unknown materials to properly evaluate how well it filters out anomalies.

Macro vs weighted avg gap

- The **macro average F1-score (0.76)** is significantly lower than the **weighted average (0.92)**. This gap is driven entirely by the "trash" category's poor performance dragging the unweighted average down. It isn't a structural failure of the model, but rather a direct reflection of trash being a highly ambiguous and visually diverse category.



Class-by-class breakdown:

- Cardboard (38 samples):** 36 correct, 2 misclassified as paper. Very clean — cardboard and paper share a similar fibrous/flat texture, so a couple of confusions is entirely expected.
- Glass (58 samples):** 53 correct, 4 $\rightarrow$ metal, 1 $\rightarrow$ plastic. The metal confusions make sense, as both glass and metal can feature highly reflective, shiny surfaces under certain lighting. No samples were falsely rejected as UNKNOWN.

- **Metal (48 samples):** 46 correct, 1 $\rightarrow$ plastic, 1 $\rightarrow$ UNKNOWN. Strong performance overall with almost no cross-class confusion; just a single misclassification into plastic and one low-confidence rejection.
- **Paper (68 samples):** 66 correct, 1 $\rightarrow$ trash, 1 $\rightarrow$ UNKNOWN. Nearly perfect — paper proves to be highly visually distinctive for the model.
- **Plastic (55 samples):** 49 correct, 2 $\rightarrow$ glass, 1 $\rightarrow$ paper, 1 $\rightarrow$ trash, 2 $\rightarrow$ UNKNOWN. The minor confusion with glass is typical for transparent or glossy plastics, while the 2 rejections point to borderline confidence on a couple of samples.
- **Trash (16 samples):** Only 8 correct out of 16 — the worst class by far. Errors are spread thin across metal (2), paper (4), plastic (1), and UNKNOWN (1). This perfectly visualizes the classification report's findings: because "trash" acts as a catch-all category with no coherent visual signature, the model frequently misclassifies it as whatever material it superficially resembles, or rejects it altogether.

Model Artifacts

Upon completion of evaluation, the following artifacts are serialised to disk for deployment:

- `svm_final_model.pkl` — The trained SVC object with probability calibration
- `svm_pca_cnn.pkl` — The fitted PCA transformer for CNN features (if PCA is used at deployment)

The rejection threshold $\tau = 0.50$ is recorded as a separate configuration parameter and must be applied consistently at inference time.

7. Discussion and Conclusions

7.1 Design Choices

Several design choices distinguish this pipeline from a standard multi-class SVM:

- **Open-Set Handling.** The unknown class is excluded from SVM training. Rather than teaching the model to classify unknowns as a seventh category (which would require representative unknown training data and biases the model toward a closed-world assumption), open-set detection is delegated to the confidence rejection layer. This is more general and does not require curated unknown samples during training.
- **PCA as an Exploratory Tool.** PCA is used only during model selection, not at deployment. This decoupling ensures the grid search remains computationally feasible while the final model operates on the full CNN feature space, retaining maximum discriminative capacity.
- **Strict Data Partitioning.** The validation set is reserved exclusively for threshold calibration. It is never used during SVM training or cross-validation. This prevents the rejection threshold from being optimistically calibrated on data that influenced model fitting.

7.2 Limitations

The choice of a fixed rejection threshold implies a static trade-off between false acceptance of unknown inputs and false rejection of known-class samples. A higher threshold produces fewer misclassifications of unknown inputs but increases the proportion of known-class samples incorrectly flagged. In operational deployment, this threshold may require recalibration as the distribution of incoming material streams evolves.

Additionally, the system assumes that the CNN features provided are of sufficient quality to discriminate among the six target material classes. The feature extraction network is treated as a fixed, pre-trained component; joint fine-tuning of the feature extractor and the SVM classifier is outside the scope of this pipeline.

7.3 Conclusions

The MSIS SVM pipeline provides a structured, principled approach to material stream classification with open-set detection. The use of CNN embeddings, PCA-accelerated model selection, and a calibrated confidence-rejection layer yields a system that is both accurate on known materials and robust to out-of-distribution inputs. The serialised model artifacts support straightforward integration into downstream deployment infrastructure.

Future work may explore adaptive rejection thresholds, integration with an online calibration mechanism, and evaluation under domain-shifted conditions representative of real-world recycling facility imagery.

7. SVM vs KNN

SVM vs KNN Comparison

Feature Set	SVM		KNN	
	Val Acc	Test Acc	Val Acc	Test Acc
HOG only	0.606	0.601	0.580	0.615
CNN only	0.864	0.922	0.867	0.883
HOG + CNN	0.835	0.908	0.885	0.887

Rejection Mechanism

Model	Method	Threshold	Accuracy with Rejection
SVM	Max class probability < 0.5 → Unknown	0.5	91.17%
KNN	Neighbor vote majority < 0.5 → Unknown	0.5	90.07%

6. Real-Time Camera Integration Protocol

1. Overview

The real-time camera application is the deployment stage of the MSI pipeline. It integrates the trained SVM classifier into a live webcam feed, classifying materials held in front of the camera and displaying results on screen instantly. The application was implemented in Python using OpenCV for video capture and TensorFlow/Keras for feature extraction.

2. Pipeline

Each frame goes through four stages:

Frame Capture: The webcam feed is captured at 640×480 pixels at 30 FPS using OpenCV's VideoCapture interface.

ROI Extraction: A fixed 250×250 pixel region of interest is cropped from the centre of each frame. A green bounding box guides the user to place the material inside this region. This reduces background interference and lowers the computational cost of feature extraction.

Feature Extraction: The cropped ROI is processed through the same pipeline used during training: resized to 224×224, passed through the frozen ResNet50 backbone with GlobalAveragePooling2D to produce a 2,048-dimensional CNN vector, scaled using the saved `cnn_scaler` and `cnn_weight` from `scaler.pkl`, then reduced with the saved PCA transform (`svm_pca_cnn.pkl`).

Classification and Rejection: The transformed vector is passed to the SVM. If the maximum class probability is at or above the rejection threshold of 0.5, the predicted class is displayed. If it falls below, the material is labelled Unknown. This is consistent with the rejection strategy used during SVM training.

3. Performance and Interface

To avoid high CPU usage from running ResNet50 on every frame, inference runs on every 10th frame only (approximately 3 predictions per second at 30 FPS). The most recent prediction is held and displayed between inference frames, keeping the video feed smooth.

The overlay displays the predicted class label in colour-coded text, a confidence percentage, and a proportional confidence bar at the bottom of the frame. Each material class has a distinct colour for fast readability. The application exits cleanly on pressing Q.

4. Required Model Files

svm_final_model.pkl Trained SVM (RBF, C=10, gamma=scale)

Svm_pca_cnn.pkl PCA fitted on CNN training features

scaler.pkl Bundle containing the CNN scaler and feature weight

label_encoder.pkl Converts numeric predictions to class name strings

Sample tests from Camera app:

